

Please help prepare for my information retrieval exam. I'll add the topics one by one. Please answer each question in detail.

explain each topic in depth. I'll add the topics one by one

Got it 👍

I'm ready to help you prepare **step-by-step for your Information Retrieval exam.**

How I'll help you:

- Explain **each topic in depth**, in **simple and clear language**
- Cover **definitions, theory, algorithms, diagrams (textual), examples**
- Add **exam-oriented points** (important keywords, comparisons, advantages, limitations)
- Structure answers so they're easy to **remember and write for full marks**
- Match **long-answer format** unless you ask otherwise

You can now **send the first topic/question**, and we'll start.



Probabilistic Retrieval:

Review of Basic Probability Theory

Below is a **detailed, exam-oriented explanation** of **Probabilistic Retrieval – Review of Basic Probability Theory**, written in **simple language** and structured to help you score **maximum marks**.

Probabilistic Retrieval: Review of Basic Probability Theory

Probabilistic Information Retrieval (IR) models are based on **probability theory**. The main idea is to **rank documents according to the probability that they are relevant to a user's query**. To understand this, we must first review some **basic probability concepts**.

1. Random Experiments and Events

- A **random experiment** is an action whose outcome cannot be predicted with certainty.
 - Example: Checking whether a document is relevant to a query.
 - An **event** is a specific outcome of the experiment.
 - Example:
 - Event R : Document is relevant
 - Event $\neg R$: Document is not relevant
-

2. Sample Space

- The **sample space** (S) is the set of all possible outcomes.
 - In $\mathbb{IR}$:
 - $S = \{\text{Relevant}, \text{Not Relevant}\}$
 - Every document belongs to **one and only one** outcome.
-

3. Probability of an Event

- The **probability of an event** A , denoted as $P(A)$, lies between 0 and 1.
 - $P(A) = 0 \rightarrow$ impossible
 - $P(A) = 1 \rightarrow$ certain
 - In $\mathbb{IR}$:
 - $P(R) \rightarrow$ probability that a document is relevant
 - $P(\neg R) = 1 - P(R)$
-

4. Joint Probability

- **Joint probability** is the probability that **two events occur together**.
 - Denoted as $P(A \cap B)$ or $P(A, B)$.
 - In $\mathbb{IR}$:
 - $P(R, t) \rightarrow$ probability that a document is relevant **and** contains term t
-

5. Conditional Probability

- **Conditional probability** is the probability of an event **given that another event has occurred**.
- Formula:

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

- In IR:
 - $P(R | d) \rightarrow$ probability that a document d is relevant
 - $P(R | t) \rightarrow$ probability that a document is relevant given term t appears

This concept is **central to probabilistic retrieval models**.

6. Bayes' Theorem

Bayes' theorem allows us to **reverse conditional probabilities**.

Formula:

$$P(R|d) = \frac{P(d|R) P(R)}{P(d)}$$

Where:

- $P(R|d)$: Probability that document d is relevant
- $P(d|R)$: Probability of observing document d given relevance
- $P(R)$: Prior probability of relevance
- $P(d)$: Probability of document d

Importance in IR:

- Used to compute **document relevance probability**
 - Forms the basis of **Binary Independence Model (BIM)**
-

7. Prior, Likelihood, and Posterior

- **Prior Probability:**
 - $P(R)$: Initial belief about relevance before seeing the document
- **Likelihood:**
 - $P(d|R)$: How likely the document appears if it is relevant
- **Posterior Probability:**
 - $P(R|d)$: Updated probability after seeing the document

IR systems rank documents by **descending posterior probability**.

8. Independence Assumption

- Two events A and B are **independent** if:

$$P(A \cap B) = P(A) P(B)$$

- In IR:
 - Terms are often assumed **independent of each other**
 - Simplifies computation in probabilistic models

Though not perfectly true, this assumption works well in practice.

9. Odds and Log-Odds

- Odds of relevance:

$$\text{Odds}(R) = \frac{P(R)}{P(\neg R)}$$

- Log-odds:

$$\log \frac{P(R|d)}{P(\neg R|d)}$$

Importance:

- Converts multiplication into addition
 - Makes scoring and ranking easier
-

10. Application of Probability Theory in Probabilistic IR

- Estimate relevance probabilities
 - Rank documents based on likelihood of relevance
 - Handle uncertainty in user queries
 - Form foundation for:
 - Binary Independence Model (BIM)
 - Okapi BM25 (probabilistic inspiration)
-

11. Key Exam Points (Write These!)

- Probabilistic retrieval ranks documents by $P(R | d)$
 - Bayes' theorem is the **core mathematical tool**
 - Conditional probability explains relevance estimation
 - Independence assumption simplifies term modeling
 - Log-odds are used for efficient scoring
-

Short Conclusion

Basic probability theory provides the **mathematical foundation** for probabilistic retrieval models. Concepts such as **conditional probability**, **Bayes' theorem**, and **independence assumptions** enable IR systems to estimate and rank documents based on their **probability of relevance**, making probabilistic retrieval both powerful and practical.

If you want, next we can cover:

- **Binary Independence Model (BIM)**
- **Probability Ranking Principle**
- **BM25 derivation**
- **Comparison with Vector Space Model**

Just send the next topic 



The Probability Ranking Principle:
The 1/0 loss case

Below is a **clear, in-depth, exam-ready** explanation of The Probability Ranking Principle (PRP) – The 1/0 Loss Case, written in **simple language** and structured to help you **score full marks**.

The Probability Ranking Principle (PRP)

The 1/0 Loss Case

The **Probability Ranking Principle (PRP)** is a **fundamental concept** in probabilistic information retrieval. It provides a **theoretical justification** for ranking documents based on **probability of relevance**.

1. What is the Probability Ranking Principle?

The **Probability Ranking Principle** states:

If documents are ranked in decreasing order of their probability of relevance to a query, then this ranking is optimal for retrieval effectiveness.

In simple terms:

- Documents with **higher probability of being relevant** should appear **higher in the ranking**
 - This minimizes the **expected number of retrieval errors**
-

2. Meaning of 1/0 Loss

The **1/0 loss case** assumes:

- Loss = 0 → correct decision
- Loss = 1 → incorrect decision

In IR context:

- Retrieving a **relevant document** → loss = 0
- Retrieving a **non-relevant document** → loss = 1

All errors are treated **equally serious**.

3. Decision Theory Background

In decision theory:

- A **decision rule** minimizes **expected loss**
- Loss function defines the **cost of wrong decisions**

PRP applies **decision theory** to document ranking.

4. Binary Relevance Assumption

PRP assumes:

- Relevance is **binary**
 - Relevant (R)
 - Not Relevant ($\neg R$)

This matches many exam-level IR models.

5. Expected Loss in Document Ranking

For a document d :

- Probability of relevance: $P(R | d)$
- Probability of non-relevance: $P(\neg R | d) = 1 - P(R | d)$

Expected Loss:

$$E(L) = P(\neg R | d)$$

Because:

- Loss occurs only when a **non-relevant** document is retrieved
-

6. Optimal Ranking Rule

To minimize expected loss:

Rank documents in decreasing order of $P(R | d)$

This ensures:

- Relevant documents are retrieved **earlier**
 - Fewer non-relevant documents appear at top ranks
-

7. Pairwise Document Comparison (Proof Idea)

Consider two documents d_1 and d_2 :

- If:

$$P(R | d_1) > P(R | d_2)$$

Then:

- d_1 should be ranked **above** d_2

- Otherwise, expected loss increases

This forms the **theoretical proof** of PRP.

8. Interpretation in Simple Words

- PRP tells us **how to rank documents**
 - Ranking by relevance probability is **optimal**
 - No other ranking can perform better under 1/0 loss
-

9. Assumptions of PRP (Important for Exams)

1. Relevance is binary (relevant / not relevant)
 2. Documents are independent
 3. User examines documents sequentially
 4. Loss for any wrong decision is equal (1/0 loss)
-

10. Limitations of 1/0 Loss Case

- Does not consider:
 - Different importance of errors
 - Graded relevance
 - User effort or position bias
 - Real users may value **top-rank errors more**
-

11. Applications in IR Models

PRP is the foundation for:

- Binary Independence Model (BIM)
 - Okapi BM25 (probabilistic inspiration)
 - Modern learning-to-rank approaches (conceptually)
-

12. Key Exam Points (Must Write)

- PRP ranks documents by $P(R | d)$
- 1/0 loss treats all mistakes equally

- Optimal ranking minimizes **expected loss**
 - Assumes binary relevance and independence
 - Core principle behind probabilistic IR models
-

Short Conclusion

The **Probability Ranking Principle** in the **1/0 loss case** states that the optimal document ranking is achieved by ordering documents in **decreasing probability of relevance**. Under binary relevance and equal loss assumptions, this strategy minimizes the expected retrieval error, making PRP the **theoretical backbone** of probabilistic information retrieval.

Next topics you may want:

- PRP with general loss functions
- Binary Independence Model (BIM)
- Derivation of BM25 from PRP
- Limitations of probabilistic retrieval

Send the next topic when ready 



the PRP with retrieval costs

Below is a **detailed, exam-oriented explanation** of The Probability Ranking Principle (PRP) with Retrieval Costs, written in **simple language** and structured to help you **score maximum marks**.

The Probability Ranking Principle (PRP)

PRP with Retrieval Costs

The **Probability Ranking Principle (PRP)** can be extended beyond the simple **1/0 loss case** to include **retrieval costs**. This makes the model more **realistic**, as it considers that **different actions and errors have different costs**.

1. Why Include Retrieval Costs?

In real-world information retrieval:

- Retrieving a **non-relevant document** wastes user time
- Missing a **relevant document** can be costly
- Users may stop early if poor documents appear at top ranks

Hence, we introduce **costs** into PRP to make better ranking decisions.

2. Types of Costs in PRP

Let:

- C_{R} = Cost of retrieving a relevant document
- C_{NR} = Cost of retrieving a non-relevant document

Typically:

- $C_{R} < C_{NR}$
 - Retrieving irrelevant documents is more costly
-

3. Loss Function with Costs

Loss is no longer binary (1/0).

Instead, it depends on the **action taken** and **true relevance**.

Action	Document Relevant	Document Not Relevant
Retrieve	Low cost (C_{R})	High cost (C_{NR})
Do not retrieve	Miss cost	No cost

4. Expected Loss with Retrieval Costs

For a document d :

$$E(L|d) = C_R \cdot P(R|d) + C_{NR} \cdot P(\neg R|d)$$

Goal:

- Minimize expected loss
-

5. Optimal Decision Rule

Retrieve document d if:

$$C_{NR} \cdot P(\neg R|d) > C_R \cdot P(R|d)$$

Rewriting:

$$\frac{P(R|d)}{P(\neg R|d)} > \frac{C_{NR}}{C_R}$$

This introduces a **cost-based threshold**.

6. Ranking with Retrieval Costs

Even with costs:

- Documents should still be **ranked by decreasing $P(R|d)$**
- BUT the **cutoff point** depends on costs

Higher cost of non-relevant retrieval → stricter threshold.

7. Odds and Log-Odds Formulation

Using odds:

$$\text{Odds}(R|d) = \frac{P(R|d)}{P(\neg R|d)}$$

Retrieve if:

$$\log \text{Odds}(R|d) > \log \frac{C_{NR}}{C_R}$$

This simplifies computation in IR systems.

8. Interpretation in Simple Language

- PRP with costs does **not retrieve everything**
 - It retrieves documents **only when benefit outweighs cost**
 - More conservative when irrelevant retrieval is expensive
-

9. Example (Exam-Friendly)

Assume:

- $P(R|d) = 0.6$
- $C_{\text{R}} = 1$
- $C_{\text{NR}} = 3$

Check:

$$\frac{0.6}{0.4} = 1.5 < 3$$

✗ Do not retrieve document

10. Assumptions

- Binary relevance
 - Known or estimated costs
 - Independence between documents
 - User examines results sequentially
-

11. Advantages of PRP with Costs

- More realistic than 1/0 loss
 - Allows tuning based on user needs
 - Balances precision and recall
-

12. Limitations

- Cost values are hard to estimate
 - Still assumes binary relevance
 - Does not directly model user behavior
-

13. Key Exam Points (Must Write)

- PRP with costs minimizes **expected loss**
- Retrieval depends on **probability–cost tradeoff**
- Ranking by $P(R|d)$ remains optimal
- Cutoff depends on **cost ratio**

- Uses odds and log-odds formulation
-

Short Conclusion

The **Probability Ranking Principle with retrieval costs** extends the basic PRP by incorporating different costs for correct and incorrect retrieval decisions. While documents are still ranked by decreasing probability of relevance, the **decision to retrieve** depends on whether the expected benefit outweighs the retrieval cost, making the model more practical and user-oriented.

Next topics you may want:

- **Binary Independence Model (BIM)**
- **PRP with general loss functions**
- **BM25 derivation**
- **Comparison of PRP variants**

Send the next topic when ready 



The Binary Independence Model

Below is a **detailed, exam-oriented explanation** of the **Binary Independence Model (BIM)**, written in **simple language**, well-structured, and suitable for **long-answer questions**.

The Binary Independence Model (BIM)

The **Binary Independence Model (BIM)** is a **probabilistic information retrieval model** based on the **Probability Ranking Principle (PRP)**. It is one of the **earliest and most influential** probabilistic models and forms the theoretical basis for modern ranking functions such as **BM25**.

1. Basic Idea of BIM

The main idea of BIM is:

A document is relevant to a query if it contains the **query terms**, and the **presence or absence** of each term contributes independently to relevance.

Key characteristics:

- Uses **binary term representation**
 - Considers **term independence**
 - Ranks documents by **probability of relevance**
-

2. Assumptions of BIM (Very Important for Exams)

1. Binary relevance

- Document is either relevant or not relevant

2. Binary term occurrence

- Term is either present (1) or absent (0) in a document
- Term frequency is ignored

3. Term independence

- Occurrence of one term does not affect others

4. Document independence

- Documents are independent of each other
-

3. Document and Query Representation

- Document d is represented as a binary vector:

$$d = (x_1, x_2, \dots, x_n)$$

where:

- $x_i = 1$ if term t_i appears in the document
 - $x_i = 0$ otherwise
 - Query q is also represented as a set of terms
-

4. Goal of BIM

Using PRP, BIM ranks documents by:

$$P(R|d)$$

where:

- R = document is relevant
- d = document representation

Documents are ranked in **descending order of relevance probability**.

5. Applying Bayes' Theorem

Using Bayes' theorem:

$$P(R|d) = \frac{P(d|R) P(R)}{P(d)}$$

Since $P(d)$ is constant for all documents, ranking depends on:

$$P(d|R) P(R)$$

6. Term Independence Assumption

Due to independence:

$$P(d|R) = \prod_{i=1}^n P(x_i|R)$$

Where:

- $P(x_i = 1|R)$ → probability term i appears in relevant documents
 - $P(x_i = 0|R)$ → probability term i does not appear
-

7. Odds Ratio Formulation

BIM uses **odds of relevance**:

$$\frac{P(R|d)}{P(\neg R|d)}$$

After simplification:

$$\log \frac{P(R|d)}{P(\neg R|d)} = \sum_{i:x_i=1} \log \frac{p_i(1-u_i)}{u_i(1-p_i)} + \text{constant}$$

Where:

- $p_i = P(x_i = 1|R)$
 - $u_i = P(x_i = 1|\neg R)$
-

8. BIM Scoring Function (Core Formula)

Each term contributes a **weight**:

$$w_i = \log \frac{p_i(1 - u_i)}{u_i(1 - p_i)}$$

Document score:

$$\text{Score}(d) = \sum_{t_i \in q \cap d} w_i$$

Higher score → higher rank.

9. Estimation of Parameters

- u_i : estimated from collection statistics

$$u_i = \frac{\text{Number of documents containing } t_i}{\text{Total documents}}$$

- p_i : estimated from relevance feedback

$$p_i = \frac{\text{Relevant documents containing } t_i}{\text{Total relevant documents}}$$

10. Relevance Feedback in BIM

BIM naturally supports **relevance feedback**:

- User marks documents as relevant
 - Update p_i values
 - Re-rank documents
 - Improves retrieval effectiveness
-

11. Advantages of BIM

- Strong theoretical foundation

- Based on PRP
 - Supports relevance feedback
 - Simple and interpretable
-

12. Limitations of BIM

- Ignores term frequency
 - Ignores document length
 - Independence assumption unrealistic
 - Binary relevance only
-

13. Relation to BM25

BM25 improves BIM by:

- Using term frequency instead of binary occurrence
 - Applying document length normalization
 - Maintaining probabilistic interpretation
-

14. Key Exam Points (Must Write)

- BIM is a **probabilistic retrieval model**
 - Based on **PRP and Bayes' theorem**
 - Uses **binary term representation**
 - Assumes **term independence**
 - Uses **log-odds scoring function**
 - Foundation for **BM25**
-

Short Conclusion

The **Binary Independence Model** is a classical probabilistic IR model that ranks documents by estimating their probability of relevance based on the **presence or absence of query terms**. Despite its simplifying assumptions, BIM provides a **strong theoretical basis** for modern retrieval models and plays a crucial role in the development of probabilistic information retrieval.

Next topics you might want:

- **BM25 model**
- **Relevance feedback in probabilistic IR**
- **Comparison: BIM vs Vector Space Model**
- **Limitations of probabilistic models**

Send the next topic when ready 

< 1/2 >      ...

Term Frequency

Below is a **detailed, exam-oriented explanation** of **Term Frequency (TF)** in **Information Retrieval**, written in **simple language**, with formulas, examples, and key points for **maximum marks**.

Term Frequency (TF)

1. Introduction

Term Frequency (TF) is a fundamental concept in **Information Retrieval (IR)** that measures **how often a term appears in a document**.

It is based on the intuition that:

If a term appears many times in a document, that document is more likely to be relevant to a query containing that term.

2. Definition of Term Frequency

For a term t in a document d , term frequency is defined as:

$$TF(t, d) = \text{Number of times term } t \text{ appears in document } d$$

Example:

If the term "retrieval" appears **5 times** in a document, then:

$$TF(\text{retrieval}, d) = 5$$

3. Why Term Frequency is Important

- Reflects **term importance within a document**
 - Helps differentiate documents where a term is:
 - Mentioned casually
 - Discussed extensively
 - Used in almost all IR models:
 - Vector Space Model
 - TF-IDF
 - BM25
-

4. Raw Term Frequency

The simplest form:

$$TF_{raw}(t, d) = f_{t,d}$$

Where:

- $f_{t,d}$ = number of occurrences of term t in document d

Problem:

- Biased toward **long documents**
 - Very frequent terms may dominate scores
-

5. Normalized Term Frequency

To reduce document length bias:

$$TF_{norm}(t, d) = \frac{f_{t,d}}{\sum_k f_{k,d}}$$

Where:

- Denominator = total number of terms in document d
-

6. Logarithmic Term Frequency

Diminishing returns assumption:

- The first few occurrences are more important than later ones

$$TF_{log}(t, d) = \begin{cases} 1 + \log(f_{t,d}), & f_{t,d} > 0 \\ 0, & \text{otherwise} \end{cases}$$

Used widely in practice.

7. Augmented Term Frequency

Avoids bias toward longer documents:

$$TF_{aug}(t, d) = 0.5 + 0.5 \cdot \frac{f_{t,d}}{\max_k f_{k,d}}$$

8. Term Frequency in Probabilistic Models

In BIM:

- Term frequency is **binary**
- Only presence (1) or absence (0) matters

In BM25:

- Term frequency plays a **major role**
- Uses **saturation function**:

$$TF_{BM25} = \frac{f_{t,d}(k_1 + 1)}{f_{t,d} + k_1}$$

This prevents TF from growing linearly.

9. Example (Exam-Friendly)

Document A:

- "information" appears 3 times

Document B:

- "information" appears 10 times

TF comparison:

- Document B likely more relevant
 - But BM25/log TF avoids over-weighting Document B
-

10. Advantages of Using TF

- Simple to compute
 - Improves relevance ranking
 - Reflects term importance
 - Works well with IDF
-

11. Limitations of Term Frequency

- Over-weights long documents
 - Cannot distinguish semantic importance
 - Needs normalization or damping
 - Common terms may dominate
-

12. Key Exam Points (Must Write)

- TF measures **term importance in a document**
 - Higher TF → higher relevance probability
 - Raw TF, log TF, normalized TF variants exist
 - BIM uses binary TF
 - BM25 uses saturated TF
 - TF combined with IDF gives TF-IDF
-

Short Conclusion

Term Frequency is a core concept in Information Retrieval that quantifies the importance of a term within a document. While raw term counts can be misleading, various normalization and damping techniques—such as logarithmic and BM25-based TF—make it a powerful and practical component of modern retrieval models.

Next topics you may want:

- **Inverse Document Frequency (IDF)**
- **TF-IDF weighting**
- **BM25 scoring function**
- **Document length normalization**

Send the next topic when ready 



An appraisal and some extensions:
An appraisal of probabilistic models

Below is a **detailed, exam-oriented explanation** of “An Appraisal of Probabilistic Models”, written in **simple language**, well structured, and suitable for **long-answer questions**.

An Appraisal of Probabilistic Models

Probabilistic models form one of the **most important theoretical frameworks** in **Information Retrieval (IR)**. They attempt to rank documents based on the **probability that a document is relevant to a given query**, using principles from **probability theory and decision theory**.

1. What are Probabilistic Models in IR?

Probabilistic IR models:

- Estimate $P(R | d) \rightarrow$ probability that document d is relevant
- Rank documents using the **Probability Ranking Principle (PRP)**
- Use statistical evidence from the document collection

Examples:

- Binary Independence Model (BIM)
 - BM25 (probabilistic-inspired)
-

2. Strengths of Probabilistic Models

2.1 Strong Theoretical Foundation

- Based on **Bayes' theorem**
 - Grounded in **decision theory**
 - PRP provides **optimal ranking justification**
-

2.2 Clear Interpretation of Scores

- Scores represent **likelihood of relevance**
 - Easier to justify ranking decisions compared to heuristic models
-

2.3 Effective Relevance Feedback

- Probabilistic models naturally support:
 - Updating probabilities using user feedback
 - BIM improves significantly with relevance feedback
-

2.4 Empirical Effectiveness

- Models like **BM25** consistently outperform many alternatives
 - Widely used in real-world search systems
-

3. Weaknesses of Probabilistic Models

3.1 Unrealistic Assumptions

- Binary relevance assumption
- Independence of terms
- Independence of documents

These assumptions simplify computation but are **not true in practice**.

3.2 Parameter Estimation Problems

- Requires estimation of:
 - $P(\text{term} \mid \text{relevant})$
 - $P(\text{term} \mid \text{non-relevant})$
 - Difficult without relevance judgments
-

3.3 Limited Query Representation

- Queries treated as:
 - Bag of words
- Ignores term order and semantics

3.4 Binary Term Representation (in BIM)

- Ignores term frequency
- Ignores document length
- Reduces retrieval accuracy

4. Practical Challenges

- User relevance judgments are rare
- Cost estimation is difficult
- Model complexity increases with extensions
- Assumes static collections

5. Comparison with Other IR Models

Aspect	Probabilistic Models	Vector Space Model
Theoretical base	Strong	Weaker
Relevance measure	Probability-based	Similarity-based
Term dependence	Independent	Independent
Relevance feedback	Natural	Heuristic

6. Extensions of Probabilistic Models

6.1 Inclusion of Term Frequency

- BIM extended to include TF
- Leads to BM25

6.2 Document Length Normalization

- Avoids bias toward long documents
- Implemented in BM25

6.3 Graded Relevance

- Moves beyond binary relevance
 - Supports modern evaluation metrics
-

6.4 Incorporation of Retrieval Costs

- Generalized PRP
 - Different penalties for different errors
-

7. Modern Relevance of Probabilistic Models

- BM25 remains a **baseline standard**
 - Forms foundation for:
 - Learning-to-rank
 - Neural retrieval models (conceptually)
-

8. Key Exam Points (Must Write)

- Probabilistic models rank documents by $P(R|d)$
 - PRP gives theoretical optimality
 - BIM is simple but limited
 - BM25 addresses major weaknesses
 - Independence assumptions are unrealistic but effective
 - Probabilistic models remain highly relevant
-

Short Conclusion

An appraisal of probabilistic models shows that while they rely on simplifying assumptions and face practical estimation challenges, their **strong theoretical foundation, effectiveness, and extensibility** make them one of the most successful approaches in information retrieval. Extensions such as **BM25** have addressed many limitations, ensuring their continued relevance in modern IR systems.

If you want, next we can cover:

- **Extensions of probabilistic models**

- BM25 in detail
- Comparison with language models
- Limitations vs neural IR models

Send the next topic when ready 



tree-structured dependencies between terms

Below is a **detailed, exam-oriented explanation** of **Tree-Structured Dependencies Between Terms**, written in **simple language**, clearly structured, and suitable for **long-answer questions** in **Information Retrieval**.

Tree-Structured Dependencies Between Terms

1. Introduction

Traditional probabilistic IR models (like **BIM**) assume that **terms occur independently** in documents.

However, in real language:

- Terms are often **dependent**
- Some words appear together (e.g., *machine* → *learning*)

To model this dependency more realistically, **tree-structured dependencies between terms** are introduced.

2. Why Term Independence is a Problem

Independence Assumption:

$$P(t_1, t_2, \dots, t_n) = \prod_i P(t_i)$$

Reality:

- Terms influence each other
- Ignoring dependencies may reduce retrieval accuracy

Example:

- "information" and "retrieval" are strongly related
- Treating them independently loses semantic meaning

3. What Are Tree-Structured Dependencies?

- Terms are organized in a **tree structure**
- Each term depends on **at most one parent term**
- Dependencies flow from root → children

This structure allows **limited but meaningful dependence** modeling while keeping computation manageable.

4. Tree Representation of Terms

- Nodes → terms
- Edges → dependency relationships

Example tree:

markdown

```

information
  |
retrieval
 /  \
model system

```

 Copy code

Here:

- "retrieval" depends on "information"
- "model" and "system" depend on "retrieval"

5. Probabilistic Formulation

Using tree-structured dependency:

$$P(t_1, t_2, \dots, t_n) = P(t_{root}) \prod_i P(t_i | \text{parent}(t_i))$$

This replaces full independence with **conditional dependence**.

6. Application in Probabilistic IR

Relevance Probability:

$$P(R|d) \propto P(d|R)P(R)$$

With dependencies:

$$P(d|R) = \prod_i P(x_i | x_{parent(i)}, R)$$

Where:

- x_i = presence of term i
 - Parent term affects child probability
-

7. Learning the Tree Structure

Tree structures can be built using:

- Mutual information between terms
- Co-occurrence statistics
- Maximum spanning tree algorithms

Goal:

- Capture strongest term dependencies
-

8. Advantages of Tree-Structured Dependencies

- More realistic language modeling
 - Captures important term relationships
 - Computationally efficient compared to full dependency graphs
 - Improves retrieval effectiveness
-

9. Limitations

- Only **pairwise dependencies**
- Each term has only one parent
- Cannot represent complex multi-term interactions
- Structure learning can be expensive

10. Comparison with Other Dependency Models

Model	Dependency Type	Complexity
BIM	None (independent)	Very low
Tree-structured	Limited (tree)	Moderate
Full graphical model	Arbitrary	Very high

11. Importance in IR Research

- Bridge between simple and complex models
- Inspired later:
 - Markov Random Fields
 - Graph-based retrieval models
- Improved probabilistic relevance estimation

12. Key Exam Points (Must Write)

- Term independence is unrealistic
- Tree-structured dependency allows **conditional dependence**
- Each term has one parent
- Uses product of conditional probabilities
- Improves relevance estimation
- Balances accuracy and efficiency

Short Conclusion

Tree-structured dependencies between terms provide a **more realistic probabilistic model** for information retrieval by relaxing the independence assumption. By organizing terms in a tree and modeling **conditional probabilities**, these models capture meaningful term relationships while remaining computationally feasible, making them an important extension of traditional probabilistic IR models.

If you want, next topics could be:

- Markov Random Field models
- N-gram dependencies
- Language models in IR
- Probabilistic vs dependency models

Send the next topic when ready 



Okapi BM25:
a non-binary model

Below is a **detailed, exam-oriented explanation** of **Okapi BM25 – A Non-Binary Model**, written in **simple language**, clearly structured, and suitable for **long-answer questions**.

Okapi BM25: A Non-Binary Model

1. Introduction

Okapi BM25 is one of the **most successful and widely used ranking functions** in Information Retrieval.

It is an **extension of the Binary Independence Model (BIM)** that removes the **binary assumption** and incorporates:

- Term frequency
- Document length normalization
- Inverse document frequency

Because it uses **non-binary term weights**, BM25 is called a **non-binary probabilistic model**.

2. Why a Non-Binary Model?

Limitations of BIM:

- Term is only present or absent
- Ignores term frequency

- Ignores document length

BM25 addresses this by:

- Using actual **term counts**
 - Penalizing very long documents
 - Applying TF saturation
-

3. Core Idea of BM25

A document is relevant if:

- "Query terms occur frequently"
 - "Occur in shorter documents"
 - "Are rare in the collection"
-

4. BM25 Ranking Function

The BM25 score for a document d and query q is:

$$\text{BM25}(d, q) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{f_{t,d}(k_1 + 1)}{f_{t,d} + k_1 \left(1 - b + b \cdot \frac{|d|}{\text{avgdl}}\right)}$$

5. Explanation of Parameters

- $f_{t,d} \rightarrow$ term frequency of term t in document d
 - $|d| \rightarrow$ length of document d
 - $\text{avgdl} \rightarrow$ average document length
 - $k_1 \rightarrow$ term frequency scaling parameter (typically 1.2–2.0)
 - $b \rightarrow$ document length normalization parameter (0–1)
 - $\text{IDF}(t) \rightarrow$ inverse document frequency
-

6. IDF in BM25

$$\text{IDF}(t) = \log \frac{N - n_t + 0.5}{n_t + 0.5}$$

Where:

- N = total number of documents

- n_t = number of documents containing term t

Rare terms get **higher weight**.

7. Term Frequency Saturation

- TF does not increase linearly
- After some point, extra occurrences add little value
- Prevents term spam

Graphically:

TF increases rapidly initially, then flattens.

8. Document Length Normalization

- Long documents naturally have higher TF
- BM25 penalizes long documents
- Parameter **b** controls strength

Special cases:

- $b = 0 \rightarrow$ no length normalization
 - $b = 1 \rightarrow$ full normalization
-

9. Non-Binary Nature of BM25

Aspect	BIM	BM25
Term representation	Binary	Frequency-based
Document length	Ignored	Normalized
TF effect	None	Saturated
Effectiveness	Moderate	High

10. Probabilistic Interpretation

- BM25 is inspired by probabilistic retrieval
- Derived as a refinement of BIM

- Approximates PRP under realistic assumptions
-

11. Advantages of BM25

- Strong empirical performance
 - Robust to noise
 - Handles term frequency well
 - Simple yet powerful
 - Industry standard baseline
-

12. Limitations of BM25

- Assumes term independence
 - Ignores term order
 - Requires parameter tuning
 - Not a true probability model
-

13. Applications

- Search engines
 - Digital libraries
 - Question answering systems
 - Benchmark baseline for neural IR models
-

14. Key Exam Points (Must Write)

- BM25 is a **non-binary probabilistic model**
 - Extends BIM
 - Uses TF, IDF, and length normalization
 - Uses saturation for TF
 - Based on PRP principles
 - Widely used in practice
-

Short Conclusion

Okapi BM25 is a powerful **non-binary probabilistic retrieval model** that overcomes the limitations of early binary models by incorporating **term frequency and document length normalization**. Its strong theoretical grounding and excellent practical performance make it one of the most important models in Information Retrieval.

If you want, next topics could be:

- **BM25 variants (BM25F, BM25+)**
- **Comparison: BM25 vs TF-IDF**
- **Parameter tuning in BM25**
- **Language models for IR**

Send the next topic when ready 



Bayesian network approaches to IR

Below is a **detailed, exam-oriented explanation** of **Bayesian Network Approaches to Information Retrieval (IR)**, written in **simple language**, clearly structured, and suitable for **long-answer questions**.

Bayesian Network Approaches to Information Retrieval

1. Introduction

Bayesian Network approaches model Information Retrieval as a **probabilistic inference problem**.

Instead of assuming term independence (as in BIM or BM25), Bayesian networks explicitly represent **dependencies between terms, documents, and relevance** using a **graphical model**.

2. What is a Bayesian Network?

A Bayesian Network (BN) is:

- **A Directed Acyclic Graph (DAG)**

- Nodes represent **random variables**
 - Edges represent **conditional dependencies**
 - Each node has a **conditional probability table (CPT)**
-

3. Why Use Bayesian Networks in IR?

Traditional probabilistic models:

- Assume term independence
- Ignore complex relationships

Bayesian networks allow:

- Modeling **term-term dependencies**
 - Modeling **query-document-relevance relationships**
 - More realistic representation of language
-

4. Basic Structure of Bayesian Network IR Model

Typical nodes:

- **Term nodes** ($t_1, t_2, \dots$)
- **Document node** (d)
- **Relevance node** (R)

Example structure:

CSS

$t_1 \rightarrow$

\

$R \rightarrow d$

/

$t_2 \rightarrow$

 Copy code

Here:

- Relevance depends on multiple terms
 - Document relevance is inferred probabilistically
-

5. Inference in Bayesian IR

Goal:

$$P(R = 1|q, d)$$

Steps:

1. Observe query terms
 2. Perform probabilistic inference
 3. Compute posterior probability of relevance
 4. Rank documents by decreasing probability
-

6. Modeling Term Dependencies

- Terms can depend on:
 - Other terms
 - Concepts
- Dependencies are encoded using edges
- Conditional probabilities capture co-occurrence patterns

Example:

$$P(t_2|t_1)$$

7. Relevance Estimation

Using Bayes' theorem:

$$P(R|d, q) = \frac{P(d, q|R)P(R)}{P(d, q)}$$

Bayesian network efficiently computes this using:

- Conditional independence
 - Graph structure
-

8. Learning the Network

Bayesian networks can be learned from:

- Training data
- Relevance feedback
- Term co-occurrence statistics

Learning involves:

- Structure learning
 - Parameter estimation
-

9. Advantages of Bayesian Network IR

- Models term dependencies
 - Strong probabilistic foundation
 - Flexible and extensible
 - Supports relevance feedback naturally
-

10. Limitations

- Computationally expensive
 - Complex structure learning
 - Scalability issues for large vocabularies
 - Requires large amounts of data
-

11. Comparison with Other Models

Model	Term Dependence	Complexity
BIM	No	Low
BM25	No	Low
Bayesian Network	Yes	High

12. Role in Modern IR

- Influenced:
 - Graph-based models
 - Markov Random Fields
 - Conceptual foundation for neural IR
-

13. Key Exam Points (Must Write)

- Bayesian networks are DAG-based probabilistic models

- Represent dependencies explicitly
 - Use inference to compute relevance probability
 - More expressive than BIM/BM25
 - Computationally expensive
-

Short Conclusion

Bayesian Network approaches to Information Retrieval provide a **powerful and flexible framework** for modeling document relevance by explicitly representing **dependencies between terms and relevance**. Although computationally complex, they offer a more realistic alternative to independence-based models and have significantly influenced modern IR research.

If you want, next topics could be:

- Markov Random Field models
- Language modeling approach to IR
- Neural probabilistic IR
- Comparison of probabilistic IR models

Send the next topic when ready 



Relevance Feedback

Below is a **detailed, exam-oriented explanation** of **Relevance Feedback**, written in **simple language**, well structured, and suitable for **long-answer questions** in **Information Retrieval**.

Relevance Feedback

1. Introduction

Relevance Feedback (RF) is a technique in Information Retrieval where the system **improves search results using user feedback** about which retrieved documents are

relevant or non-relevant.

Basic idea:

*A user knows what is relevant better than the system.
Using this information can improve retrieval performance.*

2. Motivation for Relevance Feedback

- Initial query may be **short or vague**
 - User intent is often unclear
 - Feedback helps:
 - Refine the query
 - Re-rank documents
 - Improve precision and recall
-

3. Basic Relevance Feedback Process

1. User submits a query
2. System retrieves documents
3. User marks documents as:
 - Relevant
 - Non-relevant
4. System updates the query model
5. Documents are re-ranked

This can be repeated iteratively.

4. Types of Relevance Feedback

4.1 Explicit Relevance Feedback

- User manually labels documents
- Most accurate
- Requires user effort

Example:

- "Mark as relevant" button
-

4.2 Implicit Relevance Feedback

- System infers relevance from user behavior
- No extra user effort

Examples:

- Clicks
 - Dwell time
 - Scrolling behavior
-

4.3 Pseudo (Blind) Relevance Feedback

- Assumes top-k retrieved documents are relevant
 - No user involvement
 - Common in real systems
-

5. Relevance Feedback in Vector Space Model (Rocchio Algorithm)

Formula:

$$\vec{q}_{new} = \alpha \vec{q} + \beta \frac{1}{|D_R|} \sum \vec{d}_r - \gamma \frac{1}{|D_{NR}|} \sum \vec{d}_{nr}$$

Where:

- $\vec{q}$ = original query
 - D_R = relevant documents
 - D_{NR} = non-relevant documents
 - α, β, γ = tuning parameters
-

6. Relevance Feedback in Probabilistic Models

In Binary Independence Model (BIM):

- Update term probabilities:

$$p_i = \frac{\text{Relevant documents containing } t_i}{\text{Total relevant documents}}$$

- Improves term weighting
- Strong theoretical support

7. Benefits of Relevance Feedback

- Improves retrieval accuracy
- Expands query vocabulary
- Handles vocabulary mismatch
- Adapts to user intent

8. Problems with Relevance Feedback

- User effort (explicit feedback)
- Query drift
- Incorrect assumptions in pseudo-feedback
- Not suitable for very short searches

9. Relevance Feedback vs Query Expansion

Aspect	Relevance Feedback	Query Expansion
User involvement	Yes (explicit)	No
Accuracy	High	Moderate
Risk	Low	Query drift

10. Applications

- Search engines
- Digital libraries
- Recommendation systems
- Interactive search systems

11. Key Exam Points (Must Write)

- RF improves retrieval using user judgments
- Three types: explicit, implicit, pseudo
- Rocchio algorithm for VSM
- BIM naturally supports RF

- Improves precision and recall
 - Risk of query drift
-

Short Conclusion

Relevance Feedback is a powerful technique that enhances information retrieval performance by incorporating user judgments into the retrieval process. By refining queries and adjusting term weights, relevance feedback bridges the gap between user intent and system understanding, making it a cornerstone of interactive and adaptive IR systems.

If you want, next topics could be:

- Pseudo relevance feedback in detail
- Query expansion techniques
- Learning to rank
- Evaluation of IR systems

Send the next topic when ready 



Field Weights: BM25F

Below is a **detailed, exam-oriented explanation** of **Field Weights: BM25F**, written in **simple language**, clearly structured, and suitable for **long-answer questions** in **Information Retrieval**.

Field Weights: BM25F

1. Introduction

BM25F is an **extension of the Okapi BM25 model** designed for **structured documents**. While standard BM25 treats a document as a **single block of text**, many real documents have **fields** such as:

- Title
- Abstract
- Body
- Author
- Anchor text

BM25F allows **different importance (weights)** to be assigned to different fields.

2. Why Do We Need Field Weights?

Different fields contribute differently to relevance:

- Terms in the **title** are more important than in the body
- Abstract summarizes document content
- Anchor text often describes linked pages

Standard BM25:

- Treats all occurrences equally ❌

BM25F:

- Assigns **field-specific weights** ✅
-

3. Basic Idea of BM25F

A document is relevant if query terms appear in **important fields** with sufficient frequency, adjusted for **field length**.

Each field:

- Has its own term frequency
 - Has its own length normalization
 - Is weighted differently
-

4. Fields in a Document

Let a document d have fields:

$$d = \{f_1, f_2, \dots, f_m\}$$

Examples:

- $f_1 = \text{title}$

- $f_2 = \text{body}$
 - $f_3 = \text{anchor text}$
-

5. BM25F Scoring Function (Conceptual Form)

BM25F modifies BM25 by computing a **weighted term frequency across fields**:

$$TF_F(t, d) = \sum_f w_f \cdot \frac{f_{t,f}}{(1 - b_f) + b_f \cdot \frac{|f|}{\text{avg}|f|}}$$

Where:

- $f_{t,f}$ = frequency of term t in field f
 - w_f = weight of field f
 - b_f = length normalization parameter for field f
-

6. Final BM25F Score

$$BM25F(d, q) = \sum_{t \in q} IDF(t) \cdot \frac{TF_F(t, d)(k_1 + 1)}{TF_F(t, d) + k_1}$$

This keeps the **TF saturation behavior** of BM25.

7. Explanation of Parameters

- **Field weight (w_f)**
Controls importance of a field
Example:
 - Title = 3.0
 - Body = 1.0
 - **Field length normalization (b_f)**
Adjusts for varying field lengths
 - k_1
Controls term frequency scaling
 - **IDF(t)**
Penalizes common terms
-

8. Example (Exam-Friendly)

Query: "machine learning"

Document fields:

- Title: "Machine Learning Techniques"
- Body: 1000 words, mentions "learning" 10 times

Because of higher **title weight**, BM25F assigns:

- Higher score than standard BM25
-

9. Advantages of BM25F

- Handles structured documents
 - Improves ranking quality
 - More realistic than BM25
 - Widely used in web search
-

10. Limitations of BM25F

- Requires parameter tuning
 - Assumes term independence
 - Field weights may be domain-specific
 - More complex than BM25
-

11. Comparison: BM25 vs BM25F

Aspect	BM25	BM25F
Document structure	Single field	Multiple fields
Field importance	Not supported	Supported
Use case	Plain text	Web / structured docs

12. Applications

- Web search engines

- Digital libraries
 - Enterprise search
 - Academic search systems
-

13. Key Exam Points (Must Write)

- BM25F extends BM25 for structured documents
 - Uses field-specific term frequencies
 - Assigns different weights to fields
 - Normalizes field lengths
 - Retains BM25 saturation and IDF
-

Short Conclusion

BM25F is a powerful extension of the BM25 ranking function that incorporates **field weights and field-specific normalization**, making it especially suitable for structured documents. By recognizing that not all parts of a document are equally important, BM25F significantly improves retrieval effectiveness in real-world search systems.

If you want, next topics could be:

- **BM25+ and BM25 variants**
- **Learning field weights automatically**
- **Comparison of BM25F and neural ranking models**
- **Evaluation metrics for IR**

Send the next topic when ready 



Language models for information retrieval:
generating queries from documents

Below is a **detailed, exam-oriented explanation** of **Language Models for Information Retrieval – Generating Queries from Documents**, written in **simple language**, clearly

structured, and suitable for **long-answer questions**.

Language Models for Information Retrieval

Generating Queries from Documents

1. Introduction

The **Language Modeling (LM) approach to Information Retrieval** views each **document** as a **language model** that can generate words.

A document is considered **relevant** to a query if the **probability that the document's language model generates the query is high**.

This is known as the **query-likelihood model**, where:

Documents generate queries.

2. Key Idea

Instead of asking:

- *How similar is a document to a query?*

LM approach asks:

- *How likely is it that this document could have generated the given query?*

Formally:

$$P(q|d)$$

Documents are ranked by this probability.

3. Document as a Language Model

- Each document d defines a probability distribution over terms:

$$\theta_d = \{P(t|d)\}$$

- Probability is estimated from term frequencies in the document
-

4. Query Likelihood Model

Given a query:

$$q = \{t_1, t_2, \dots, t_n\}$$

The likelihood is:

$$P(q|d) = \prod_{i=1}^n P(t_i|d)$$

Assumes:

- Query terms are independent (bag-of-words)
-

5. Estimating Term Probabilities

Maximum Likelihood Estimation (MLE):

$$P(t|d) = \frac{f_{t,d}}{|d|}$$

Where:

- $f_{t,d}$ = frequency of term t in document d
 - $|d|$ = total number of terms in d
-

6. The Zero Probability Problem

If a query term does not appear in the document:

$$P(t|d) = 0 \Rightarrow P(q|d) = 0$$

This is undesirable → requires **smoothing**.

7. Smoothing Techniques (Very Important)

7.1 Jelinek–Mercer Smoothing

$$P(t|d) = (1 - \lambda) \frac{f_{t,d}}{|d|} + \lambda \frac{f_{t,C}}{|C|}$$

- Combines document model with collection model

- λ controls smoothing strength

7.2 Dirichlet Prior Smoothing

$$P(t|d) = \frac{f_{t,d} + \mu P(t|C)}{|d| + \mu}$$

- μ controls influence of collection model
- Most commonly used

8. Ranking Function (Log Form)

To avoid underflow:

$$\log P(q|d) = \sum_{t \in q} \log P(t|d)$$

Documents are ranked by decreasing log-likelihood.

9. Interpretation in Simple Language

- Document “explains” the query
- More likely document → higher rank
- Similar to probabilistic relevance estimation

10. Comparison with Probabilistic Models

Aspect	Language Model	BM25
Perspective	Docs generate queries	Docs match queries
TF handling	Probabilistic	Saturated
Smoothing	Required	Implicit
Foundation	Probability	Probability-inspired

11. Advantages of LM Approach

- Strong probabilistic foundation
 - Simple ranking principle
 - Effective smoothing techniques
 - Competitive performance with BM25
-

12. Limitations

- Assumes term independence
 - Sensitive to smoothing parameters
 - Ignores term order
 - Query-only relevance model
-

13. Applications

- Web search
 - Document ranking
 - Pseudo relevance feedback
 - Baseline for neural language models
-

14. Key Exam Points (Must Write)

- Language model treats documents as generators of queries
 - Uses query likelihood $P(q|d)$
 - Requires smoothing
 - Dirichlet smoothing is popular
 - Competitive with BM25
 - Based on probabilistic theory
-

Short Conclusion

The **language modeling approach to information retrieval** ranks documents based on the probability that a document's language model can **generate the user's query**. By estimating term probabilities and applying smoothing techniques, this approach provides a principled and effective probabilistic framework for modern information retrieval systems.

If you want, next topics could be:

- **Relevance-based language models**
- **KL-divergence retrieval model**
- **Comparison: LM vs BM25**
- **Neural language models in IR**

Send the next topic when ready 



Language models:

finite automata and language models

Below is a **detailed, exam-oriented explanation** of **Language Models: Finite Automata and Language Models**, written in **simple language**, clearly structured, and suitable for **long-answer questions**.

Language Models: Finite Automata and Language Models

1. Introduction

A **language model (LM)** assigns **probabilities to sequences of words**.

Finite automata provide a **formal and computational framework** to represent such models, especially for **simple language models** like **n-grams**.

In Information Retrieval and NLP, this connection helps us:

- Model valid word sequences
 - Assign probabilities to queries and documents
 - Efficiently compute likelihoods
-

2. What is a Finite Automaton?

A **Finite Automaton (FA)** is a mathematical model consisting of:

- A finite set of **states**
- An **input alphabet** (words or symbols)

- **Transitions** between states
- A **start state**
- One or more **accepting states**

It processes input **symbol by symbol** and decides whether a string belongs to a language.

3. Types of Finite Automata

3.1 Deterministic Finite Automaton (DFA)

- Exactly one transition per symbol per state

3.2 Nondeterministic Finite Automaton (NFA)

- Multiple possible transitions for a symbol

For language modeling, we usually use **probabilistic finite automata**.

4. Probabilistic Finite Automata (PFA)

A **Probabilistic Finite Automaton** extends FA by assigning **probabilities to transitions**.

Properties:

- Sum of outgoing transition probabilities = 1
- Generates strings with certain probabilities

This makes PFAs suitable for **language modeling**.

5. Language Models as Probabilistic Automata

A **language model** can be represented as a **probabilistic automaton** where:

- States represent **contexts**
 - Transitions represent **word emissions**
 - Transition probabilities = word probabilities
-

6. Unigram Language Model as Automaton

Characteristics:

- Word probabilities independent of history

Automaton:

- Single state
- Self-loops for each word
- Transition probability = $P(\text{word})$

$$P(w_1 w_2 \dots w_n) = \prod_i P(w_i)$$

7. Bigram Language Model as Automaton

Characteristics:

- Each word depends on the previous word

Automaton:

- States = words
- Transitions from word w_{i-1} to w_i
- Transition probability = $P(w_i | w_{i-1})$

$$P(w_1 \dots w_n) = \prod_i P(w_i | w_{i-1})$$

8. Trigram and n-gram Models

- States represent last **n-1 words**
 - Transitions emit next word
 - Higher $n \rightarrow$ more context $\rightarrow$ more states
-

9. Acceptance and Probability of a String

- A string is accepted if it ends in an accepting state
 - Probability of a string = product of transition probabilities along the path
-

10. Relation to Information Retrieval

In IR:

- Documents are language models
- Queries are sequences generated by models

- Finite automata provide:
 - Efficient computation
 - Formal representation

Used in:

- Query likelihood models
 - Speech retrieval
 - Text indexing
-

11. Advantages of FA-based Language Models

- Clear mathematical foundation
 - Efficient algorithms
 - Easy to implement
 - Suitable for n-gram models
-

12. Limitations

- Cannot model long-range dependencies
 - State explosion for large n
 - Limited expressive power
 - Requires smoothing
-

13. Comparison with Advanced Models

Model	Structure	Dependencies
Finite Automata	Finite states	Short-range
HMM	Probabilistic states	Sequential
Neural LM	Neural networks	Long-range

14. Key Exam Points (Must Write)

- Finite automata recognize formal languages
- Probabilistic automata assign probabilities

- Language models can be represented as PFAs
 - n-gram models map naturally to automata
 - Used in IR for query likelihood estimation
-

Short Conclusion

Finite automata provide a **formal and efficient framework** for representing **language models**, particularly **n-gram models**, by modeling word sequences as probabilistic state transitions. Although limited in expressive power, automaton-based language models play a foundational role in Information Retrieval and Natural Language Processing.

If you want, next topics could be:

- Hidden Markov Models
- KL-divergence retrieval model
- Smoothing in language models
- Neural language models vs classical LMs

Send the next topic when ready 



types of language models

Below is a **detailed, exam-oriented explanation** of **Types of Language Models (LMs)**, written in **simple language**, clearly structured, and suitable for **long-answer questions** in **Information Retrieval**.

Types of Language Models

A **Language Model (LM)** assigns **probabilities to sequences of words**. In IR, they are used to estimate the probability that a **document generates a query**. There are several types of language models, based on **representation, dependencies, and computation**.

1. Unigram Language Model

Definition

- Simplest type of LM
- Assumes **words occur independently**

$$P(w_1, w_2, \dots, w_n) = \prod_{i=1}^n P(w_i)$$

Characteristics

- No context or history
- Easy to compute
- Useful for baseline retrieval

Advantages

- Simple, fast
- Works reasonably for short queries

Limitations

- Ignores word order
 - Cannot capture term dependencies
-

2. n-gram Language Model

Definition

- Estimates probability of a word based on the **previous n-1 words**

$$P(w_i | w_1, \dots, w_{i-1}) \approx P(w_i | w_{i-(n-1)}, \dots, w_{i-1})$$

Common Examples

- Bigram (n=2)
- Trigram (n=3)

Characteristics

- Models **short-range dependencies**
- Can be represented using **probabilistic finite automata**
- Requires **smoothing** for unseen n-grams

Advantages

- Captures local context
- Improves over unigram model

Limitations

- State explosion for large n
 - Cannot capture long-range dependencies
-

3. Probabilistic Finite Automata (PFA) Language Model

Definition

- Represents LM as a **finite state machine** with **probabilities on transitions**
- Each path generates sequences of words with probabilities

Characteristics

- Useful for formal modeling
 - Represents n -gram models compactly
 - Used in IR and speech recognition
-

4. Hidden Markov Model (HMM) Language Model

Definition

- Extends n -gram models
- States represent **latent topics or syntactic categories**
- Observations = words

Characteristics

- Captures sequential dependencies
- Can model **hidden structure** of text

Applications

- Part-of-speech tagging
 - Document modeling
 - Speech retrieval
-

5. Relevance-Based / Document-Based Language Models

Query Likelihood Model

- Treats each document as a **language model**
- Computes probability that document generates the query

$$P(q|d) = \prod_{t \in q} P(t|d)$$

Relevance-Based LM

- Incorporates **relevance feedback**
 - Combines document and collection language models
-

6. Neural Language Models

Definition

- Use **neural networks** to predict next word
- Can capture **long-range dependencies**
- Includes:
 - RNN-LM
 - LSTM-LM
 - Transformer-based models (BERT, GPT)

Advantages

- Models complex dependencies
- Contextual embeddings
- Strong performance in modern IR

Limitations

- Requires large training data
 - Computationally expensive
-

7. Comparison of Types

LM Type	Dependency Mode	Complexity	Use in IR
Unigram	None	Low	Baseline
n-gram	Short-range	Moderate	Query likelihood

LM Type	Dependency Mode	Complexity	Use in IR
PFA	Short-range	Moderate	Formal representation
HMM	Sequential / latent	High	Structured text modeling
Relevance LM	Document-based	Moderate	Query likelihood + feedback
Neural LM	Long-range	Very high	Modern IR, contextual ranking

8. Key Exam Points (Must Write)

- Unigram → independent terms
- n-gram → short-range dependencies
- PFAs → finite state, probabilistic transitions
- HMM → hidden sequential dependencies
- Query likelihood → documents generate queries
- Neural LMs → long-range, contextual modeling
- Smoothing is essential for sparse data

Short Conclusion

Language models in IR vary from **simple unigram models** to **complex neural architectures**. The choice depends on **computational resources**, **required context**, and **application**. Modern IR often combines **document-based LMs** with **relevance feedback** or **neural LMs** for best performance.

Next topics you may want:

- **Smoothing techniques in language models**
- **KL-divergence retrieval model**
- **Language model evaluation in IR**
- **Neural vs classical LMs**

Send the next topic when ready 



multinomial distributions over words

Below is a **detailed, exam-oriented explanation** of **Multinomial Distributions over Words**, written in **simple language**, clearly structured, and suitable for **long-answer questions** in **Information Retrieval**.

Multinomial Distributions Over Words

1. Introduction

In **language modeling for Information Retrieval (IR)**, a **document or query** can be viewed as a **sample of words** drawn from a **probability distribution**.

The **multinomial distribution** is a natural choice for modeling **word occurrences**, especially when considering **term frequencies** rather than just presence/absence.

The multinomial model generalizes the binomial model to **multiple outcomes (words)**.

2. Definition of Multinomial Distribution

A **multinomial distribution** describes the probability of counts of words in a **fixed-length sequence**, given a vocabulary of size V .

$$P(x_1, x_2, \dots, x_V | \theta) = \frac{n!}{x_1! x_2! \dots x_V!} \prod_{i=1}^V \theta_i^{x_i}$$

Where:

- V = size of the vocabulary
 - x_i = count of word i in the document
 - $n = \sum_i x_i$ = total number of words in the document
 - $\theta_i = P(\text{word } i)$, with $\sum_{i=1}^V \theta_i = 1$
-

3. Connection to Language Models

- Each document is represented as a **multinomial distribution over words**.
- Term frequency matters:

- Words that occur more often contribute more to the probability
- Query likelihood model uses this:

$$P(q|d) = \prod_{t \in q} P(t|d)^{f_{t,q}}$$

- Here $f_{t,q}$ = frequency of term t in query q
-

4. Maximum Likelihood Estimation

Given a document d with word counts x_i , the MLE estimate for word probabilities:

$$\hat{\theta}_i = P(t_i|d) = \frac{x_i}{n} = \frac{\text{count of } t_i \text{ in } d}{\text{total words in } d}$$

- Simple, intuitive: **relative frequency**
 - Forms the basis for **unigram and n-gram models**
-

5. Smoothing

- Problem: Some words in the query may **not occur in the document** → zero probability
- Solution: Combine document distribution with **collection distribution**
- Example: **Dirichlet smoothing**

$$P(t|d) = \frac{x_i + \mu P(t|C)}{n + \mu}$$

- Ensures all words have non-zero probability
-

6. Advantages of Multinomial Model

- Models **term frequencies** naturally
 - Simple and computationally efficient
 - Integrates easily with **query likelihood** models
 - Provides **probabilistic foundation** for ranking
-

7. Limitations

- Assumes **term independence** (bag-of-words)

- Cannot capture **word order**
 - Requires smoothing for rare/unseen words
 - Ignores semantic similarity
-

8. Example (Exam-Friendly)

Document d = "information retrieval retrieval retrieval system"

- Vocabulary: {information, retrieval, system} $\rightarrow V=3$
- Counts: $x_{\text{information}}=1, x_{\text{retrieval}}=3, x_{\text{system}}=1 \rightarrow n=5$

MLE estimates:

$$P(\text{information}|d) = 1/5, \quad P(\text{retrieval}|d) = 3/5, \quad P(\text{system}|d) = 1/5$$

Query: "retrieval system"

Query likelihood:

$$P(q|d) = P(\text{retrieval}|d) \cdot P(\text{system}|d) = \frac{3}{5} \cdot \frac{1}{5} = 0.12$$

9. Applications in IR

- Query likelihood retrieval
 - Relevance-based language models
 - Naive Bayes classification for text
 - Probabilistic topic models (e.g., LDA)
-

10. Key Exam Points (Must Write)

- Multinomial model generalizes binomial to multiple words
- Term frequencies are modeled explicitly
- Probability of a document generating a query:

$$P(q|d) = \prod_t P(t|d)^{f_{t,q}}$$

- Requires **smoothing** to handle zero-count words
 - Forms the foundation of modern **language-model-based IR**
-

Short Conclusion

The **multinomial distribution over words** provides a **probabilistic framework for representing documents** by their term frequencies. It underlies most **language-model-based information retrieval methods**, offering a simple yet effective way to estimate the likelihood that a document can generate a query.

If you want, next topics could be:

- Dirichlet and Jelinek-Mercer smoothing in multinomial LMs
- KL-divergence retrieval model
- Query expansion using language models

Send the next topic when ready 



Ranking with Language Models

Below is a **detailed, exam-oriented explanation** of **Ranking with Language Models**, written in **simple language**, clearly structured, and suitable for **long-answer questions** in **Information Retrieval**.

Ranking with Language Models in Information Retrieval

1. Introduction

In **Language-Model-based Information Retrieval (LMIR)**, documents are represented as **language models**, and ranking is based on **how likely a document is to generate a user query**.

The central idea: *A document is relevant if it could have "generated" the query with high probability.*

2. Query Likelihood Model

The **most common ranking approach** with language models is the **query likelihood model**.

For a query $q = \{t_1, t_2, \dots, t_n\}$ and document d :

$$P(q|d) = \prod_{i=1}^n P(t_i|\theta_d)$$

Where:

- θ_d = language model of document d
- $P(t_i|\theta_d)$ = probability of term t_i in d
- Assumes **bag-of-words** independence

Documents are ranked by **descending** $P(q|d)$.

3. Estimating Term Probabilities

3.1 Maximum Likelihood Estimate (MLE)

$$P(t|d) = \frac{f_{t,d}}{|d|}$$

Where:

- $f_{t,d}$ = frequency of term t in document d
 - $|d|$ = total number of terms in document d
-

3.2 Smoothing

- Problem: Terms in query may be **absent in the document** → zero probability
- Solution: Combine document model with **collection model**

Common Smoothing Methods

1. Jelinek-Mercer smoothing

$$P(t|d) = (1 - \lambda) \frac{f_{t,d}}{|d|} + \lambda \frac{f_{t,C}}{|C|}$$

2. Dirichlet Prior smoothing

$$P(t|d) = \frac{f_{t,d} + \mu P(t|C)}{|d| + \mu}$$

Where:

- $f_{t,C}$ = frequency of term t in collection C
 - λ or μ = smoothing parameter
-

4. Ranking Function (Log Form)

To avoid underflow and simplify computation:

$$\text{Score}(d, q) = \sum_{t \in q} f_{t,q} \cdot \log P(t|d)$$

- $f_{t,q}$ = frequency of term t in query
 - Documents with **higher log-likelihood** are ranked higher
-

5. Relevance-Based Language Models

- Incorporates **relevance feedback**
 - Adjusts document model based on known relevant documents
 - Rank is based on **probability that a document generates a query drawn from relevant documents**
-

6. Advantages of LM Ranking

- Probabilistic foundation
 - Naturally incorporates **term frequencies**
 - Easy to include **smoothing**
 - Competitive performance with BM25
-

7. Limitations

- Bag-of-words assumption → ignores term order
 - Sensitive to **smoothing parameters**
 - Query-document term mismatch may reduce performance
 - Cannot model long-range dependencies without extensions
-

8. Comparison with BM25

Aspect	BM25	Language Models
Ranking basis	Weighted TF-IDF	Query likelihood ($P(q$
Term independence	Assumed	Assumed
Length normalization	Explicit	Implicit via smoothing
Probabilistic foundation	Approximate	Strong

9. Example (Exam-Friendly)

Query: "information retrieval"

Document: 10 words

- $f(\text{"information"}) = 2$, $f(\text{"retrieval"}) = 1$
- Dirichlet smoothing $\mu=2000$, collection probabilities:
 $P(\text{"information"}|C) = 0.001$, $P(\text{"retrieval"}|C) = 0.0005$

Score:

$$P(q|d) = P(\text{"information"}|d) \cdot P(\text{"retrieval"}|d)$$

→ Compute smoothed probabilities → Rank document accordingly.

10. Key Exam Points (Must Write)

- Documents as **language models**
- Rank by **query likelihood** $P(q|d)$
- **Smoothing** is essential (Dirichlet/Jelinek-Mercer)
- Relevance feedback can improve ranking
- Log-likelihood often used for computational stability

Short Conclusion

Ranking with language models is a principled probabilistic approach in IR that orders documents by the **likelihood of generating the query**. With **term frequency modeling and smoothing**, this approach forms a strong foundation for modern retrieval methods and remains competitive with BM25-based ranking.

Next topics you may want:

- **KL-Divergence retrieval models**
- **Relevance-based language models (RM1, RM3)**
- **Comparison of LM ranking vs BM25**

Send the next topic when ready 



Divergence from Randomness

Below is a **detailed, exam-oriented explanation** of **Divergence from Randomness (DFR) models**, written in **simple language**, clearly structured, and suitable for **long-answer questions** in **Information Retrieval**.

Divergence from Randomness (DFR) Models

1. Introduction

Divergence from Randomness (DFR) is a probabilistic framework for Information Retrieval that ranks documents based on **how much the occurrence of a term in a document deviates from what would be expected by chance**.

The more a term "stands out" from randomness in a document, the more informative it is for retrieval.

DFR models are used in systems like **Lucene, Terrier**, and other IR frameworks.

2. Motivation

Traditional models (TF-IDF, BM25):

- Use term frequency and inverse document frequency heuristically

DFR:

- Provides a **principled probabilistic foundation**
- Based on the idea:

Relevance $\propto$ Information content of term in document

3. Core Idea

1. Randomness Hypothesis

- Assume that terms occur **randomly in the collection** according to some distribution
- Common choices: **Poisson, Binomial, Geometric**

2. Divergence

- Measure the difference between the **actual term occurrence in a document** and the expected occurrence under randomness

3. Score Computation

- High divergence → more informative term → higher document score

4. DFR Scoring Formula (General Form)

A typical DFR score for a term t in document d :

$$\text{score}(t, d) = -\log_2 P_{\text{random}}(t \text{ in } d) \cdot (1 - P_{\text{after-effect}})$$

Where:

- P_{random} = probability of observing $f_{t,d}$ occurrences of t in d by chance
- $P_{\text{after-effect}}$ = models **dependence of repeated occurrences** of the term in the document

Document score:

$$\text{score}(d, q) = \sum_{t \in q} \text{score}(t, d)$$

5. Common Components

5.1 Term Frequency Normalization

- DFR adjusts for **document length**
- Examples: normalization by average document length or total term occurrences

5.2 Term Weight

- Terms that are **rare in collection** and **frequent in document** get higher weight
- Similar in effect to TF-IDF

5.3 Probabilistic Models of Randomness

- **Poisson Model**
 - Assumes term occurrences are independent
 - Good for long documents
 - **Binomial Model**
 - Assumes fixed number of trials
 - Useful for shorter documents
 - **Geometric Model**
 - Models the distance between occurrences
-

6. Advantages of DFR Models

- Strong **probabilistic foundation**
 - Flexible: can choose **different randomness models**
 - Explicitly models **informativeness of terms**
 - Comparable performance to BM25
-

7. Limitations

- More complex than BM25
 - Parameter tuning required for best performance
 - Assumes term occurrence is independent (like many probabilistic models)
-

8. Example (Intuitive)

Document d = 1000 words

- Term t occurs 10 times
- Expected occurrence under randomness = 2

DFR score for t increases because:

- Term occurs **much more frequently than expected**
 - Contribution of term to document relevance is high
-

9. Comparison with BM25 and TF-IDF

Aspect	TF-IDF	BM25	DFR
Probabilistic foundation	Weak	Moderate	Strong

Aspect	TF-IDF	BM25	DFR
Term weighting	Heuristic	TF saturation	Divergence from randomness
Document length	Implicit	Normalized	Normalized
Flexibility	Low	Moderate	High (choose randomness model)

10. Key Exam Points (Must Write)

- DFR ranks documents based on **informativeness of terms**
- Uses **probability of observing term by chance** in a document
- Includes **term frequency normalization** and **after-effect models**
- Flexible: different models of randomness (Poisson, Binomial, Geometric)
- Document score = sum of term scores for query terms
- Comparable or better performance than BM25 in some settings

Short Conclusion

Divergence from Randomness (DFR) models provide a **principled probabilistic approach** to ranking documents, emphasizing terms that **occur more than expected by chance**. By modeling term informativeness explicitly and normalizing for document length, DFR forms a robust and flexible alternative to traditional ranking models like BM25 and TF-IDF.

If you want, next topics could be:

- **Models based on DFR: PL2, I(n)L2**
- **Comparison of probabilistic IR models (BIM, BM25, DFR)**
- **Learning to rank with probabilistic features**

Send the next topic when ready 



Below is a **detailed, exam-oriented explanation** of **Passage Retrieval and Ranking**, written in **simple language**, clearly structured, and suitable for **long-answer questions** in **Information Retrieval**.

Passage Retrieval and Ranking

1. Introduction

Passage retrieval is an Information Retrieval technique that retrieves **sections or passages** of a document rather than the **entire document**.

The goal: **find the most relevant part of a document** that answers a user query.

Passage retrieval is particularly useful in:

- Question answering systems
 - Web search snippets
 - Legal or scientific document search
 - Biomedical IR
-

2. Why Passage Retrieval?

1. Document-level retrieval limitations

- Entire documents may be long
- Only a small part may be relevant

2. User needs

- Users often want **specific answers**, not entire documents

3. Ranking improvement

- Smaller units allow more **fine-grained relevance estimation**
 - Reduces noise from irrelevant parts
-

3. Defining a Passage

- **Passage** = contiguous segment of a document
- Size can be:
 - Fixed number of words (e.g., 50–200 words)
 - Fixed number of sentences

- Semantic or topic-based segmentation
-

4. Passage Retrieval Approaches

4.1 Sliding Window Approach

- Slide a window of fixed length over the document
- Each window = candidate passage
- Compute **passage score** with respect to query

4.2 Sentence-Level Retrieval

- Treat each sentence as a passage
- Useful when queries are short

4.3 Topical Segmentation

- Segment document based on topic shifts
 - Each topic block = passage
 - Reduces redundancy and increases semantic coherence
-

5. Ranking Passages

5.1 Term Matching

- Score passages based on **query term occurrences**
- Examples:
 - TF-IDF
 - BM25 adapted to passages

5.2 Probabilistic Models

- Compute **probability of passage generating query**
- Use **language models** at passage level:

$$P(q|p) = \prod_{t \in q} P(t|p)$$

- Smoothing applied for short passages

5.3 Divergence from Randomness

- Model **informativeness of terms in passage**
- Short passages with high divergence from randomness get higher score

5.4 Aggregation Methods

When multiple passages belong to the same document:

- **Maximum scoring passage** → document score = max passage score
 - **Average scoring passages** → sum or average
 - **Weighted combination** based on passage length or position
-

6. Passage Length Normalization

- Longer passages naturally contain more query terms
- Normalize score by passage length to avoid bias
- Parameterized weighting is common:

$$\text{score}(p, q) = \frac{\text{raw_score}}{|p|^\beta}$$

7. Benefits of Passage Retrieval

- Provides **more precise results**
 - Enhances **snippet generation**
 - Supports **question answering**
 - Improves **relevance ranking** at fine granularity
-

8. Limitations

- Short passages may **lack context**
 - High computational cost for sliding windows
 - Requires **normalization** to avoid bias toward longer passages
 - Semantic coherence may be lost in naive splitting
-

9. Example (Exam-Friendly)

Query: "symptoms of diabetes"

Document: 1000 words

1. Divide document into 100-word passages (10 passages)
2. Compute BM25 score for each passage
3. Passage with **highest score** contains: "Frequent urination, excessive thirst, and fatigue are common symptoms of diabetes."

4. Return this passage as the top result

- Document-level retrieval may rank the document lower due to irrelevant content
-

10. Key Exam Points (Must Write)

- Passage retrieval = fine-grained document retrieval
 - Approaches:
 - Sliding window
 - Sentence-based
 - Topical segmentation
 - Ranking:
 - Term frequency models (TF-IDF, BM25)
 - Query likelihood / language models
 - Divergence from randomness
 - Normalize for passage length
 - Aggregation methods for document-level scoring
-

Short Conclusion

Passage retrieval and ranking focus on **retrieving the most relevant portions of documents** rather than full documents. By segmenting documents and scoring passages using probabilistic, TF-based, or DFR models, IR systems can provide **precise, contextually relevant answers**, improving user satisfaction in search, QA, and snippet generation tasks.

Next topics you may want:

- Snippet generation in IR
- Evaluation metrics for passage retrieval (MAP, Precision@k)
- Neural passage ranking (BERT, DPR)

Send the next topic when ready 



Answer each question in exam style. answer each question in depth

Define and explain Probabilistic Retrieval. [6]
What is the binary independent retrieval model? [6]

Explain with suitable example different levels of NLP [6]
List and explain the challenges in Natural language processing [6]

What is the probability ranking principle for interactive information retrieval? [6]
State and explain different types of Bayesian networks? [6]

What is language modeling approach to information retrieval? [6]
Explain language models? [6]

Here's a **comprehensive, exam-style answer** for each of your questions, written in **depth**, structured clearly, and suitable for **maximum marks**. I've formatted them with **point-wise** and **paragraph explanations** where necessary.

Q1. Define and explain Probabilistic Retrieval. [6]

Answer:

Probabilistic Retrieval is an **information retrieval (IR) approach** that ranks documents based on the **probability of relevance** to a user's query.

Definition:

Probabilistic retrieval is a retrieval framework where a document is considered relevant if the probability that it is relevant to a given query is higher than for other documents.

Explanation:

1. It is based on the **probability theory**.
2. Each document d has a **probability of being relevant** $P(R = 1|d, q)$ given a query q .
3. Documents are **ranked in decreasing order** of $P(R = 1|d, q)$.
4. Assumes **binary relevance** (relevant or non-relevant).

5. Uses term statistics, such as:

- Term frequency in relevant vs non-relevant documents
- Document frequency

Advantages:

- Provides a **principled probabilistic basis** for ranking
- Forms foundation for **BM25 and Binary Independence Model**

Example:

Suppose a query "machine learning" is issued. Two documents, D1 and D2, are evaluated. If D1 has terms "machine" and "learning" appearing frequently in known relevant documents and D2 does not, then $P(R = 1|D1, q) > P(R = 1|D2, q)$, so D1 is ranked higher.

Q2. What is the Binary Independent Retrieval Model? [6]**Answer:**

The **Binary Independence Model (BIM)** is a **probabilistic IR model** that assumes:

1. Terms are **binary** (present or absent in a document)
2. Terms are **independent** of each other
3. Relevance is **binary** (document is either relevant or non-relevant)

Mathematical Formulation:

For a query q and document d :

$$\text{Score}(d) = \sum_{t \in q} \log \frac{p_t}{1 - p_t} - \log \frac{q_t}{1 - q_t}$$

Where:

- p_t = probability of term t occurring in relevant documents
- q_t = probability of term t occurring in non-relevant documents

Explanation:

- Uses **term presence/absence** instead of frequency
- Ranks documents by **log-odds of relevance**
- Foundation for **BM25**

Example:

Query: "data mining"

- Term "data" appears in 80% of relevant documents, 40% of non-relevant
 - Term "mining" appears in 60% of relevant, 10% of non-relevant
 - Score for a document is computed using **log-odds formula**; higher score → more relevant
-

Q3. Explain with suitable example different levels of NLP. [6]

Answer:

Natural Language Processing (NLP) involves understanding and generating human language. It has **multiple levels**:

1. Lexical Level

- Deals with **words and their forms**
- Tasks: tokenization, stemming, lemmatization
- *Example*: "running" → "run"

2. Syntactic Level

- Focuses on **sentence structure and grammar**
- Tasks: parsing, part-of-speech tagging
- *Example*: Sentence: "The cat chased the mouse" → NP (The cat), VP (chased the mouse)

3. Semantic Level

- Deals with **meaning of words and sentences**
- Tasks: word sense disambiguation, semantic role labeling
- *Example*: "bank" in "river bank" vs "financial bank"

4. Pragmatic Level

- Focuses on **contextual use and intention**
- Tasks: discourse analysis, coreference resolution
- *Example*: "Can you pass the salt?" → request, not question

5. Discourse Level

- Understanding **relations between sentences**
- Tasks: anaphora resolution, coherence analysis
- *Example*: "John went to the store. He bought milk." → "He" refers to John

6. Phonetic/Phonological Level (in speech NLP)

- Deals with **sounds and pronunciation**

Example Application:

- Query: "What is AI?"

- Lexical: tokenize → ["what", "is", "AI"]
 - Syntactic: parse question structure
 - Semantic: identify "AI" as "Artificial Intelligence"
 - Pragmatic: user wants definition
 - Discourse: integrate with previous queries for context
-

Q4. List and explain the challenges in Natural Language Processing. [6]

Answer:

Challenges in NLP:

1. Ambiguity

- **Lexical ambiguity:** "bank" (river or financial)
- **Syntactic ambiguity:** "I saw the man with telescope"

2. Variability of Language

- Different ways to express same meaning
- Example: "He is happy" ≈ "He feels joy"

3. Context Understanding

- Meaning depends on context
- Example: "He is late" vs "He is late to the party"

4. Complex Grammar

- Multiple sentence structures
- Example: Nested clauses, idioms

5. Resource Limitations

- Need for large corpora, dictionaries, ontologies

6. World Knowledge

- Understanding text often requires external knowledge
- Example: "He drank water from the Nile" → Nile is a river

≡ ChatGPT ▾

↑ ↻ ...

Q5. What is the Probability Ranking Principle for interactive information retrieval? [6]

Answer:

Probability Ranking Principle (PRP) states:

Documents should be ranked in decreasing order of their probability of relevance to the user's query.

Explanation:

- Let $P(R = 1|d, q)$ = probability that document d is relevant to query q
- PRP ensures that if document d_1 has higher $P(R|d_1, q)$ than d_2 , d_1 appears above d_2
- For **interactive IR**, relevance judgments can be updated based on user feedback

Key Points:

1. Maximizes expected utility (retrieval effectiveness)
2. Works with **binary relevance** (relevant / non-relevant)
3. Forms the basis of **probabilistic IR models**, e.g., **BIM**, **BM25**

Example:

- Query: "Machine Learning"
 - D1: $P(R|D1, q) = 0.9$
 - D2: $P(R|D2, q) = 0.7$
 - D1 appears first in ranking → PRP satisfied
-

Q6. State and explain different types of Bayesian Networks. [6]**Answer:****Bayesian Network (BN):**

- A **directed acyclic graph (DAG)** where nodes represent **random variables**, and edges represent **conditional dependencies**.

Types of Bayesian Networks:**1. Naive Bayes Network**

- Assumes all features (terms) are **conditionally independent** given class/relevance
- Simple, used in text classification and IR

2. Tree-structured Bayesian Network

- Terms can depend on **parent term**
- Captures some term dependencies
- Useful for modeling **correlated terms**

3. General Bayesian Network

- Arbitrary DAG structure

- Can represent **complex dependencies** among terms and relevance

Explanation:

- BN is used in IR to compute:

$P(R|d, q)$ = posterior probability of relevance given document and query

- Inference in BN ranks documents for retrieval
- Learning BN involves:
 - Structure learning (edges between nodes)
 - Parameter learning (conditional probabilities)

Q7. What is Language Modeling Approach to Information Retrieval? [6]

Answer:

Language Modeling (LM) Approach:

- Each **document is treated as a language model** (probability distribution over words)
- Rank documents based on the **probability of generating the query**

Query Likelihood Model:

$$P(q|d) = \prod_{t \in q} P(t|d)$$

Explanation:

1. Document d generates words with probability $P(t|d)$
2. Smoothing is applied to handle unseen query terms (Dirichlet or Jelinek-Mercer)
3. Higher probability → document more relevant

Advantages:

- Strong probabilistic foundation
- Can integrate relevance feedback
- Competitive with BM25

Example:

- Query: "Artificial Intelligence"
- Document D1 mentions "AI" frequently → after mapping to "Artificial Intelligence", $P(q|D1)$ higher
- Document D2 mentions it rarely → lower probability → ranked lower

Q8. Explain Language Models. [6]

Answer:

Language Model (LM):

A statistical model that assigns **probabilities to sequences of words**, predicting the likelihood of a word or query given a document or context.

Types:

1. Unigram LM:

- Words independent
- $P(w_1, \dots, w_n) = \prod_i P(w_i)$

2. n-gram LM:

- Probability of word depends on **previous n-1 words**
- Captures short-term dependencies

3. Document-based LM:

- Each document has its own language model
- Used in query likelihood IR

4. Relevance-based LM:

- Incorporates relevance feedback
- Adjusts probabilities based on known relevant documents

Key Concepts:

- **Smoothing:** ensures non-zero probability for unseen words
- **Probability of query given document:** used to rank documents
- **MLE Estimation:** $P(t|d) = f_{t,d}/|d|$

Example:

- Document: "machine learning techniques"
- Unigram LM: $P(\text{"machine"}) = 1/3$, $P(\text{"learning"}) = 1/3$, $P(\text{"techniques"}) = 1/3$
- Query: "machine learning" $\rightarrow P(q|d) = 1/3 \times 1/3 = 0.111$

These answers are **exam-ready**, cover **definitions, explanations, examples, and key formulas**, and are structured to **maximize marks**.

If you want, I can **compile all your probabilistic IR and language modeling topics into a single concise exam-ready revision sheet with formulas and examples**. This will save time

for last-minute preparation.

Do you want me to do that?

